Laboratorul 6. Informații invizibile: forensics și steganografie

fișă de laborator · 2 × 90 min (cu pauză la mijloc) · două lucrări · se lucrează individual, la terminal

> **CE CONSTRUIȚI ASTĂZI.** Două unelte de forensics, scrise de voi în Python, care intră în arsenal. Lucrarea A este un decoder universal: dat un șir, recunoaște stratul (base64, hex, URL), îl desface, iar dacă e un XOR cu cheie scurtă sau un hash de parolă, îl sparge prin forță brută sau dicționar. Lucrarea B este forensics pe fișiere: citești semnătura reală, scoți textul și metadatele (inclusiv GPS) dintr-o poză, extragi un fișier ascuns, apoi ascunzi și dezvălui un mesaj prin steganografie LSB. Amândouă se predau cum cere regula de fier: cod și output, niciodată o captură de ecran.

> **ȚINTE PERMISE ȘI CADRUL LEGAL.** Lucrați DOAR pe fișierele pregătite pentru laborator (le generați voi la pasul de pregătire) și pe propriile voastre fișiere. Forensics pe dispozitivul, contul sau fișierele altcuiva, fără acordul lui, este acces neautorizat la date, o infracțiune (Cod penal RM, capitolul XI). Aceleași unelte, pe ce vă aparține, sunt muncă legală. Diferența nu este unealta, este permisiunea.

Pregătire (comună celor două lucrări)

Intră în depozitul disciplinei, activează mediul virtual și instalează biblioteca de imagini. Apoi rulează scriptul de pregătire de mai jos: el îți creează, chiar la tine, fișierele pe care le vei analiza (un șir codat în straturi, un mesaj XOR, o poză cu EXIF și GPS, și o poză cu un fișier ascuns). Astfel toată lumea lucrează pe aceleași ținte, repetabil.

```
cd tehnici-prog-<nume>
# activeaza mediul (Windows PowerShell):
.\.venv\Scripts\Activate.ps1
# sau Linux:  source .venv/bin/activate

pip install pillow
mkdir -p arsenal/forensics teste probe

# creeaza fisierele de laborator:
python probe/pregateste.py     # scrie in probe/
```

Scriptul probe/pregateste.py îți este dat (în depozitul de start al laboratorului). El produce: probe/mesaj.txt (șir în mai multe straturi de codare), probe/xor.bin (text criptat cu XOR pe un octet), probe/hashuri.txt (câteva hash-uri de spart), probe/foto.jpg (cu EXIF și GPS) și probe/ascuns.png (cu o arhivă lipită la coadă). Dacă lipsește, cere-l asistentului.

Ce poate crăpa. Dacă `import PIL` dă eroare, biblioteca se numește la instalare `pillow` (nu „PIL"). Dacă nu vezi `(.venv)` în prompt, mediul nu e activat.

LUCRAREA A (nucleu). Decoderul universal: arsenal/forensics/decode.py

Obiectiv: un program care primește un șir sau un fișier, recunoaște stratul și îl desface; iar pentru un XOR cu cheie de un octet sau un hash de parolă, îl sparge singur. Pornești de la scheletul de mai jos; funcțiile marcate cu TODO sunt ale tale.

A1. Text și octeți: încălzirea (10 min)

Creează arsenal/forensics/decode.py. Fixează-ți întâi diferența dintre text și octeți, de care atârnă tot.

```
s = "Salut"
b = s.encode()          # bytes
print(b, b[0])          # b"Salut" 83
# TODO: tipareste b.hex() si refa bytes din hex
#       cu bytes.fromhex(...)
```

De observat. `b[0]` e un număr, nu o literă. Tot ce urmează lucrează pe `bytes`, nu pe text.

A2. Recunoști stratul după formă (14 min)

Scrie o funcție care, dat un șir, ghicește ce codare e, uitându-se la caractere. Nu decoda încă, doar clasifică.

```
import re
def ghici_strat(s):
    # TODO: daca are %xx -> "url"
    # TODO: daca doar 0-9 a-f si lungime para -> "hex"
    # TODO: daca doar A-Za-z0-9+/= -> "base64"
    return "necunoscut"
print(ghici_strat("%2Fetc"))   # url
print(ghici_strat("48656c6c6f"))  # hex
```

De observat. Ordinea contează: verifică întâi „%" (URL), apoi hexul (mai strict), apoi base64 (cel mai larg alfabet).

A3. Desfaci stratul recunoscut (14 min)

Acum chiar decodează, în funcție de ce a zis A2. Întoarce și tipul, și rezultatul.

```
import base64
from urllib.parse import unquote
def desfa(s):
    strat = ghici_strat(s)
    # TODO: pentru fiecare strat, decodeaza
    #   base64 -> base64.b64decode(s)
    #   hex    -> bytes.fromhex(s)
    #   url    -> unquote(s).encode()
    # intoarce (strat, rezultat_bytes)
    return strat, s.encode()
```

A4. Straturi multiple, într-o buclă (14 min)

Un rezultat poate fi tot o codare. Decodează în buclă până iese text citibil (sau te oprești după câteva încercări, ca să nu intri în buclă infinită).

```
def citibil(b):
    # TODO: True daca toti octetii sunt 32..126
    return all(32 <= c < 127 for c in b)

val = "NTM1OTU3NTk="
for _ in range(8):
    strat, b = desfa(val)
    # TODO: daca citibil(b): opreste si tipareste
    #       altfel val = b.decode(); continua
```

A5. Spargi un XOR cu cheie de un octet (16 min)

Citește probe/xor.bin (octeți), încearcă toate cele 256 de chei și păstrează rezultatul care arată a text.

```
date = open("probe/xor.bin", "rb").read()
for k in range(256):
    clar = bytes(b ^ k for b in date)
    # TODO: daca citibil(clar):
    #       print("cheie:", k, clar.decode())
```

De observat. Operatorul ^ e XOR; `b ^ k` aplică cheia pe un octet. O cheie de un octet are fix 256 de valori, deci le încerci pe toate.

A6. Spargi un hash prin dicționar (16 min)

Din probe/hashuri.txt ia un hash. Identifică algoritmul după lungime, apoi caută parola într-o wordlist.

```
import hashlib
tinta = open("probe/hashuri.txt").readline().strip()
print(len(tinta))   # 32 md5 / 40 sha1 / 64 sha256
for cuv in open("probe/wordlist.txt", encoding="latin1"):
    cuv = cuv.strip()
    # TODO: daca hashlib.md5(cuv.encode()).hexdigest()
    #       == tinta: print("gasit:", cuv); break
```

De observat. Nu „decriptezi" hash-ul: hashuiești fiecare cuvânt și compari. Lungimea îți spune ce algoritm să folosești.

A7. argparse, fișa JSON și un test (16 min)

Leagă totul într-o unealtă comandată din afară, salvează rezultatul ca fișă, și scrie un test.

```
import argparse, json
p = argparse.ArgumentParser()
p.add_argument("intrare")          # sir sau cale
p.add_argument("--fisier", action="store_true")
a = p.parse_args()
# TODO: daca a.fisier, citeste octetii din cale;
#       altfel trateaza intrarea ca sir
# TODO: salveaza {strat, rezultat} in
#       arsenal/forensics/fisa.json
```

Un test scurt care ține decoderul cinstit:

```
# teste/test_decode.py
from arsenal.forensics.decode import desfa
def test_base64():
    strat, b = desfa("U2FsdXQ=")
    assert strat == "base64"
    assert b == b"Salut"
```

Predai: `decode.py`, `fisa.json`, `teste/test_decode.py`, plus în `raport.md`: ce ascundea mesajul cu straturi, ce cheie XOR ai găsit și ce parolă ai spart.

LUCRAREA B (extindere). Forensics pe fișiere și un mesaj ascuns

Obiectiv: citești ce spun fișierele despre ele (semnătură, text, metadate), extragi un fișier ascuns, apoi ascunzi și dezvălui un mesaj prin steganografie LSB. Se lucrează pe fișierele din probe/.

B1. Magic bytes: tipul real, nu extensia (12 min)

Deschide un fișier în mod binar și citește-i primii octeți. Compară-i cu semnăturile cunoscute.

```
def tip_real(cale):
    cap = open(cale, "rb").read(8)
    # TODO: daca cap incepe cu b"%PDF" -> "pdf"
    #       b"\x89PNG" -> "png" ; b"\xff\xd8" -> "jpeg"
    #       b"PK\x03\x04" -> "zip/docx"
    return cap.hex()
print(tip_real("probe/foto.jpg"))
```

De observat. Modul `"rb"` nu e opțional pentru fișiere binare: fără el, octeții se pot altera.

B2. strings: textul citibil dintr-un binar (12 min)

Scoate secvențele de caractere imprimabile dintr-un fișier, cu un regex pe octeți.

```
import re
date = open("probe/ascuns.png", "rb").read()
# TODO: for m in re.findall(rb"[ -~]{4,}", date):
#           print(m.decode())
```

```
# cauta un indiciu: o cale, un mesaj, un nume de fisier
```

B3. Metadate EXIF și GPS din poză (16 min)

Citește blocul EXIF din probe/foto.jpg și scoate coordonatele GPS. Convertește-le în grade zecimale.

```
from PIL import Image
from PIL.ExifTags import TAGS
exif = Image.open("probe/foto.jpg")._getexif() or {}
for id, val in exif.items():
    print(TAGS.get(id, id), "=", val)
# TODO: ia blocul GPS (tag 34853) si tipareste
#       latitudinea si longitudinea
```

De observat. Coordonatele vin ca (grade, minute, secunde) plus emisfera. Le poți pune pe o hartă: de aici valoarea lor pentru OSINT, și motivul să ștergi EXIF înainte de a publica.

B4. File carving: scoți fișierul ascuns (14 min)

probe/ascuns.png are o arhivă ZIP lipită la coadă. Găsește-i semnătura și salveaz-o separat.

```
date = open("probe/ascuns.png", "rb").read()
i = date.find(b"PK\x03\x04")
# TODO: daca i != -1:
#   open("gasit.zip","wb").write(date[i:])
#   print("arhiva extrasa de la octetul", i)
# apoi dezarhiveaza si vezi ce era inauntru
```

B5. Ascunzi un mesaj prin LSB (16 min)

Scrie un mesaj în ultimii biți ai pixelilor dintr-o poză gazdă. Rezultatul arată identic cu originalul.

```
from PIL import Image
img = Image.open("probe/foto.jpg").convert("RGB")
px = list(img.getdata())
mesaj = b"FLAG{ascuns_in_pixeli}"
biti = "".join(f"{x:08b}" for x in mesaj)
# TODO: pentru fiecare bit, ia un canal si pune
#   canal = (canal & ~1) | int(bit)
# TODO: img.putdata(px_nou); img.save("stego.png")
```

De observat. Salvează în `.png` (fără pierdere); un `.jpg` recompresează imaginea și distruge biții ascunși.

B6. Dezvălui mesajul înapoi (14 min)

Citește ultimii biți ai pixelilor din stego.png, regrupează-i câte 8 și reconstituie mesajul.

```
px = list(Image.open("stego.png").convert("RGB").getdata())
biti = ""
for (r, g, b) in px:
    # TODO: aduna ultimii biti (r&1, g&1, b&1)
    pass
# TODO: grupeaza cate 8 biti -> octeti -> text
#       opreste-te la primul octet 0 (final de mesaj)
```

De observat. Dezvăluirea e exact inversul ascunderii: aceiași pași, citiți invers. Când apare text citibil, ai găsit mesajul.

> **PENTRU VÂRFUL SĂLII (opțional).** Dacă termini devreme, în probe/ găsești challenge.bin: un fișier care combină tot ce ai făcut azi (un strat de encoding peste un XOR, cu un hash de verificat). Despică-l singur, fără schelet, și pune pașii în raport. Este genul de problemă pe care o vei întâlni pe pwn.college și pe challenge-urile de forensics.

Predai: scripturile de la B1-B6, fișierul `stego.png`, și în `raport.md`: tipul real al fișierelor, ce ai găsit în metadate (inclusiv GPS), ce era în arhiva ascunsă, și mesajul LSB dezvăluit.

Ce predai și cum se notează

L	i	v	r	a	b	i	l
`arsenal/forensics/decode.py`	Decoderul: recunoastere, desfacere, straturi, XOR brute, hash (A2-A7).						
`arsenal/forensics/fisa.json`	Fisa unui mesaj decodat, salvata de unealta (A7).						
`teste/test_decode.py`	Testul care trece (A7).						
forensics.py` (sau similar)	Scriptul cu B1-B4: magic bytes, strings, EXIF/GPS, carving.						
stego.png` + scriptul LSB	Poza cu mesaj ascuns si codul care ascunde/dezvaluie (B5-B6).						
raport.md` (secțiune nouă)	Ce ai decodat, ce cheie/parola ai spart, metadate GPS, arhiva ascunsa, mesajul LSB.						

Cum predai (obligatoriu, prin git):

Urci TOT codul în depozitul tău git tehnici-prog-<nume> (cadrul didactic este colaborator) și dai commit + push la finalul lucrării. Asistentul îl descarcă de acolo (git pull) și îl rulează. Nu se trimite nimic pe e-mail și nicio captură de ecran.

```
git add .
git commit -m "Lab 6: decoder universal + forensics si stego"
git push
```

Nucleul (Lucrarea A) se face pe viu, în laborator. Extinderea (Lucrarea B) se poate continua acasă: fișierele din probe/ sunt generate de tine, deci nu depinzi de rețeaua laboratorului. Se notează codul care rulează și output-ul, nu capturi de ecran.